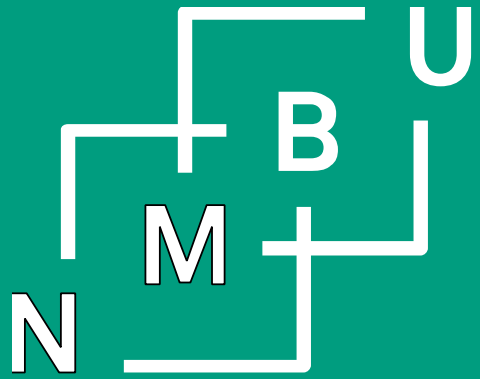


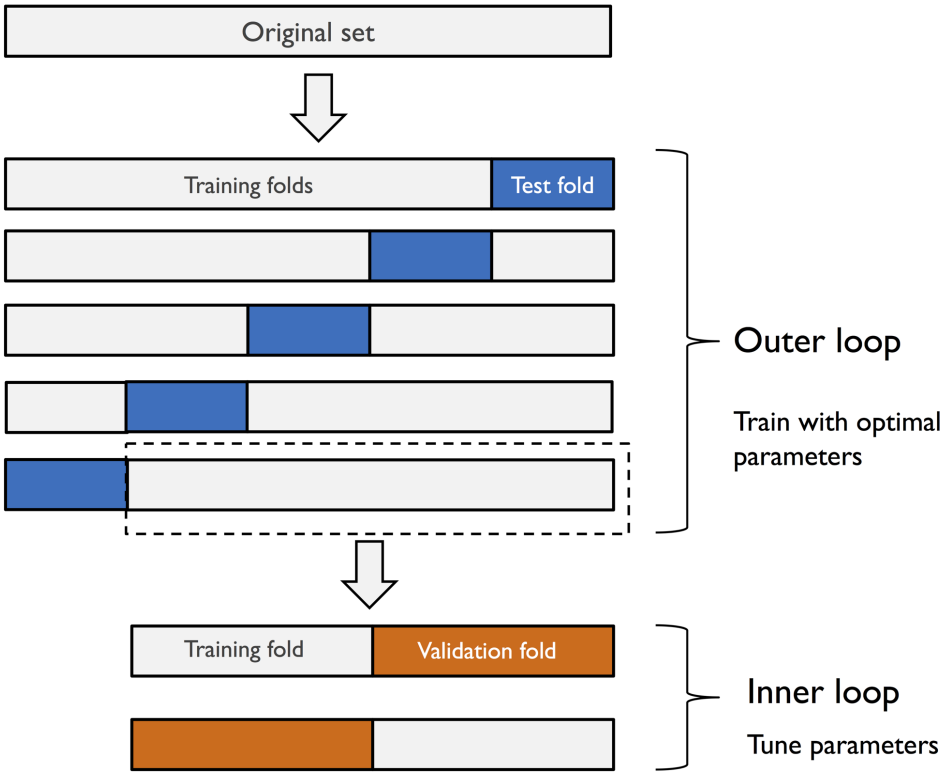
Norwegian University
of Life Sciences

Cross-validation and parameter optimisation

Chapter 6 in the Python Machine Learning book

Nested cross-validation

- A cross-validation loop within a cross-validation loop
- Addresses the fact that the model is also sensitive to how the initial split between the training/validation set and test set is done
- Becomes very computationally expensive
- Is rarely done when we are working with large datasets



Jupyter: Chapter 6, part 2a

Nested cross-validation

- **Nested cross-validation** is a robust model evaluation technique that addresses bias in model performance estimates, especially when hyperparameter tuning is involved.

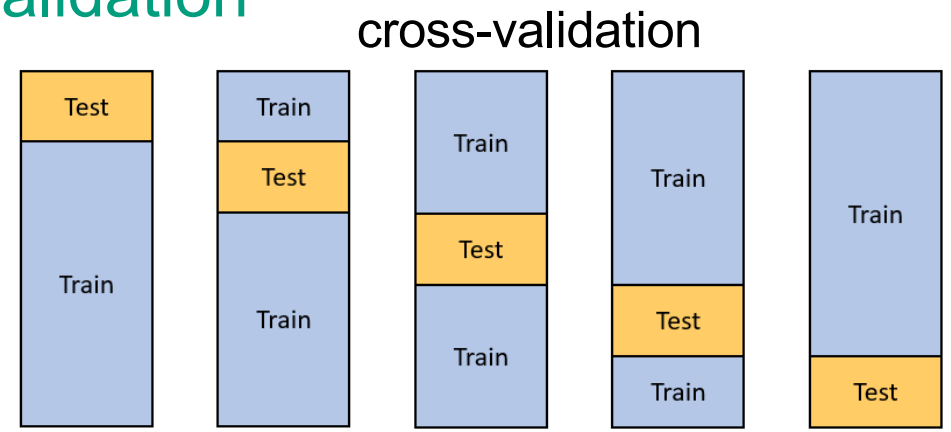
How?

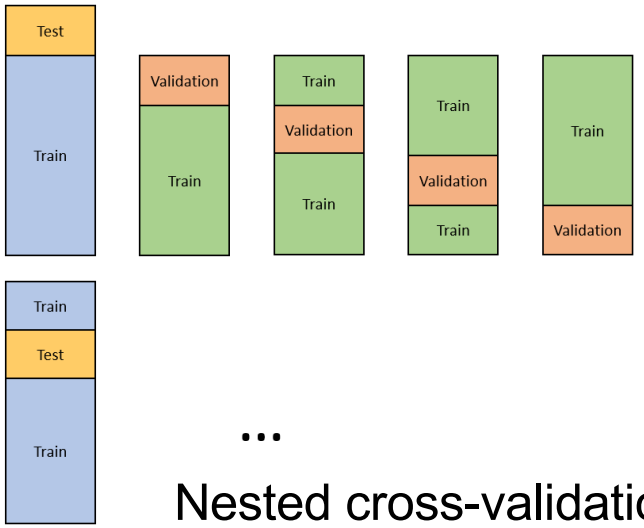
- Outer loop (model evaluation):
  - Splits the dataset into training and testing sets (e.g., 5 folds).
  - Each fold is treated as a separate testing set for evaluating the model's final performance.
- Inner loop (hyperparameter tuning):
  - Within each training fold of the outer loop, the model's hyperparameters are tuned using cross-validation (e.g., 5 folds).
  - The goal is to select the best hyperparameters using training data without ever touching the outer test set.

Source: https://www.analyticsvidhya.com/blog/2021/03/a-step-by-step-guide-to-nested-cross-validation/

Advantages of nested cross-validation

- Unbiased performance estimation:
  - Since the outer test set is completely independent of the inner tuning process, the performance estimate is much more reliable.
  - **Better generalization:** Prevents overfitting on hyperparameters, leading to models that generalize better to unseen data.
  - **Reliable model selection**: Ensures you are choosing the best model based on truly unseen data, mimicking real-world deployment scenarios.

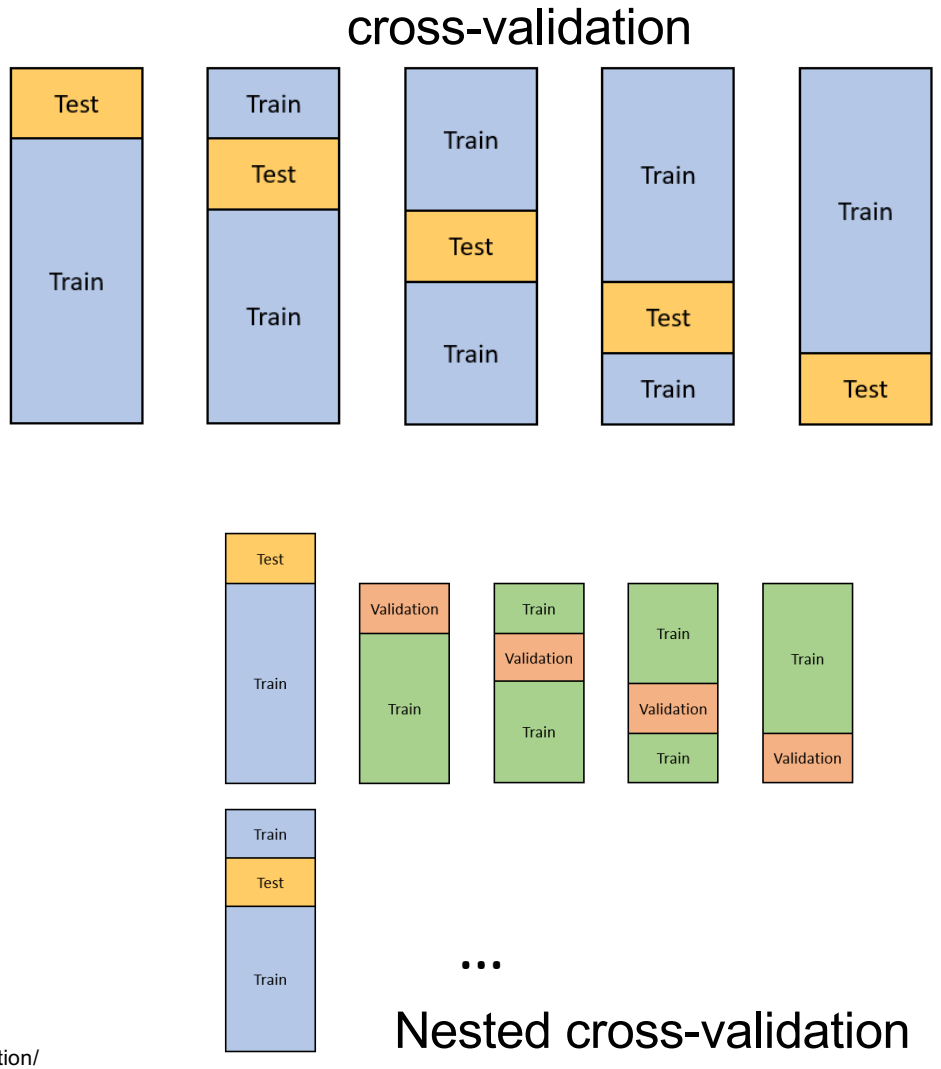




Source: https://www.analyticsvidhya.com/blog/2021/03/a-step-by-step-guide-to-nested-cross-validation/

Why not just use regular cross-validation?

- Regular cross-validation (CV) is fine for simple models with no hyperparameter tuning. However, when hyperparameter tuning is involved, it becomes biased and overly optimistic because of:
  - **Data leakage:** If you tune hyperparameters and evaluate performance on the same test set, your results are not reliable.
  - **Overfitting risk:** You may accidentally select hyperparameters that perform well on the test set by luck rather than generalization.



Source: https://www.analyticsvidhya.com/blog/2021/03/a-step-by-step-guide-to-nested-cross-validation/

Binary classifier example

- Rare type of asthma which occurs in approximately 1% of the population
- Can be deadly, but is a condition that is easy to treat
- Assume that you have two binary classifiers:
  - C1 with an accuracy of 99%
  - C2 with an accuracy of 98%
- Which one would you prefer?

Binary classifier example

- Let us take a close look at how the two classifiers perform on the test set
- 1000 samples in total, 990 healthy and 10 with the disease
- C1 makes 10 mistakes, C2 makes 18 mistakes
- Does anyone see any issue with C1, despite it being more accurate than C2?

IDX	1	2	3	...	973	974	975	976	977	978	979	980	981	982	983	984		
GT	0	0	0	...	0	0	0	0	0	0	0	0	0	0	0	0	No disease	Disease
C1	0	0	0	...	0	0	0	0	0	0	0	0	0	0	0	0	Correct	Incorrect
C2	0	0	0	...	0	1	1	1	1	1	1	1	1	1	1	1	Correct	Incorrect

IDX	985	986	987	988	989	990	991	992	993	994	995	996	997	998	999	1000		
GT	0	0	0	0	0	0	1	1	1	1	1	1	1	1	1	1	No disease	Disease
C1	0	0	0	0	0	1	0	0	0	0	0	0	0	0	0	1	Correct	Incorrect
C2	1	1	1	1	1	1	0	1	1	1	1	1	1	1	1	1	Correct	Incorrect

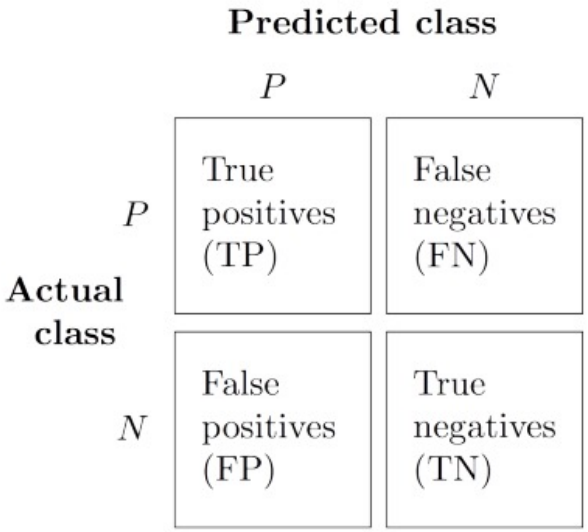
Confusion matrix

- Often an overall accuracy is not enough, so we want to get a better idea of how well a classifier detects specific classes
- To get a more thorough understanding of a model´s predicted performance we look at a model´s **confusion matrix**
- For binary classifiers it looks like the matrix to the right

		Predicted class: P	Predicted class: N
Actual class	P	True positives (TP)	False negatives (FN)
	N	False positives (FP)	True negatives (TN)

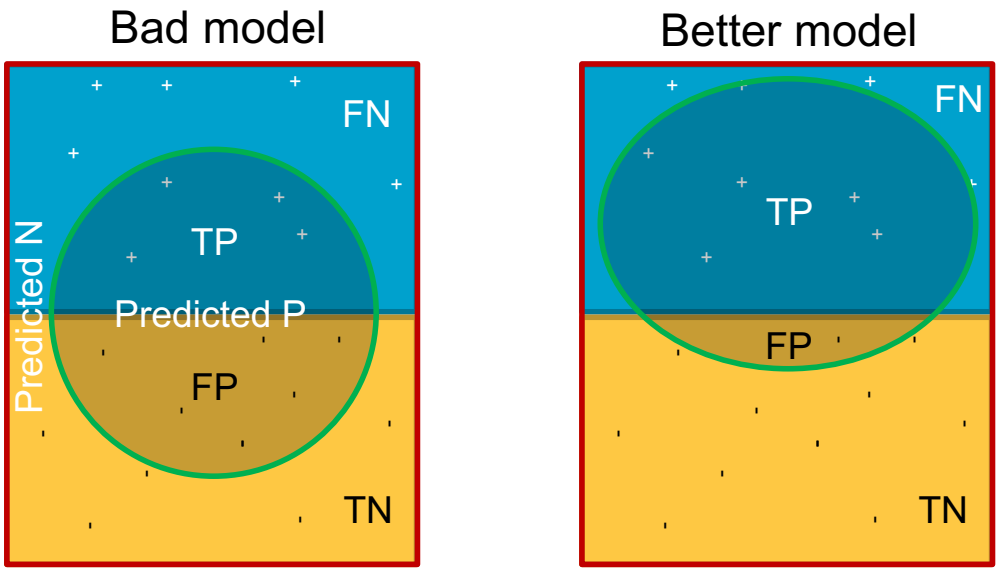
Confusion matrix

- It is a square matrix showing the classes that a model predicts versus the classes of the ground truth



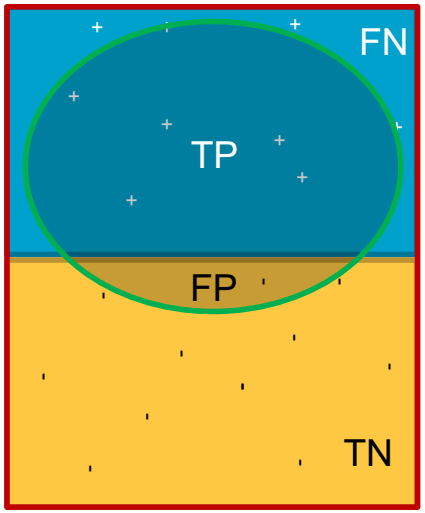
- **TP**: Positive sample, model predicts positive
- **FP**: Negative sample, model predicts positive
- **FN**: Positive sample, model predicts negative
- **TN**: Negative sample, model predicts negative

Confusion matrix

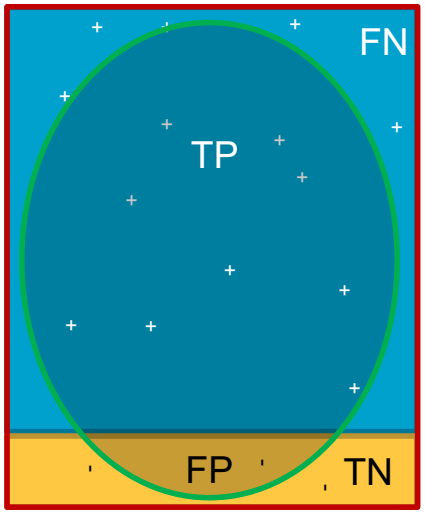


Confusion matrix

Balanced classes



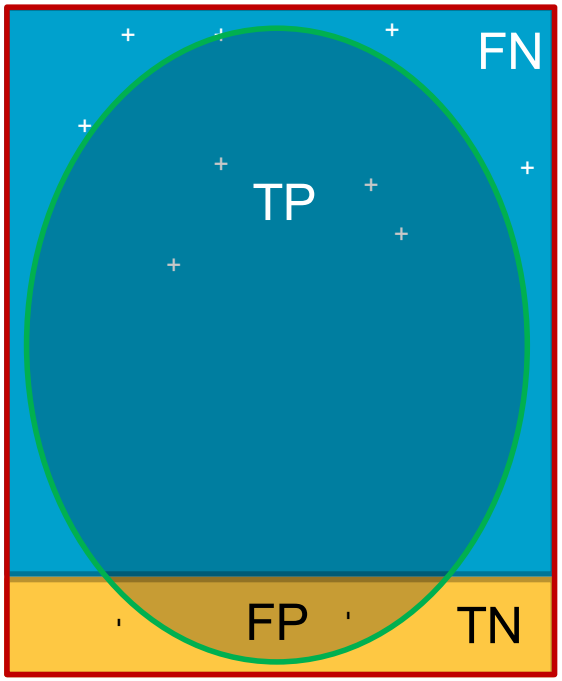
Unbalanced classes



Risk of predicting
everything positive

Class imbalance

- High degree of imbalance:
  - Low performance of predictors
  - Skewed scoring metrics
    - Doctor´s response to very rare diseases:
      - "You are healthy!"
    - Choose right metric, use 'macro' setting
- When fitting models:
  - Different weighting of samples from different groups, `class_weight='balanced'`
  - Upsample smaller class(es) `resample()`
  - Downsample larger class(es)
  - Generate synthetic samples



Confusion matrix

- Let´s compute the confusion matrix of C1 and C2 from the rare asthma condition example

	Predicted class: P	Predicted class: N
Actual class: P	True positives (TP)	False negatives (FN)
Actual class: N	False positives (FP)	True negatives (TN)

IDX	1	2	3	...	973	974	975	976	977	978	979	980	981	982	983	984		
GT	0	0	0	...	0	0	0	0	0	0	0	0	0	0	0	0	No disease	Disease
C1	0	0	0	...	0	0	0	0	0	0	0	0	0	0	0	0	Correct	Incorrect
C2	0	0	0	...	0	1	1	1	1	1	1	1	1	1	1	1	Correct	Incorrect

IDX	985	986	987	988	989	990	991	992	993	994	995	996	997	998	999	1000		
GT	0	0	0	0	0	0	1	1	1	1	1	1	1	1	1	1	No disease	Disease
C1	0	0	0	0	0	1	0	0	0	0	0	0	0	0	0	1	Correct	Incorrect
C2	1	1	1	1	1	1	0	1	1	1	1	1	1	1	1	1	Correct	Incorrect

	PP	*PN*
P	TP	FN
N	FP	TN

C1

	PP	*PN*
P	1	9
N	1	989

C2

	PP	*PN*
P	9	1
N	17	973

Different evaluation metrics

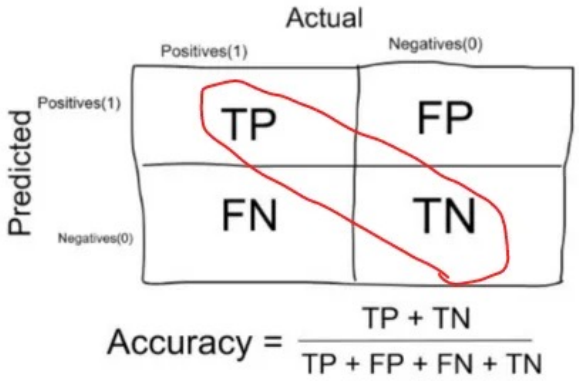
- As of now you should be familiar with two evaluation metrics:
  - **Accuracy (ACC)**
  - **Prediction error (ERR)**
- **ACC**: Number of **correct** predictions divided by the total number of predictions
- **ERR**: Number of **incorrect** predictions divided by the total number of predictions

$$\mathbf{ACC} = \frac{TP + TN}{TP + TN + FP + FN}$$

$$\mathbf{ERR} = \frac{FP + FN}{TP + TN + FP + FN} = 1 - \mathbf{ACC}$$

Performance metrics

- **Accuracy:** Measures the overall correctness of the model



Source: https://medium.com/@MohammedS/performance-metrics-for-classification-problems-in-machine-learning-part-i-b085d432082b

Different evaluation metrics

- Different evaluation metrics are important for different classification problems
- True Positive Rate (**TPR**), False Positive Rate (**FPR**), False Negative Rate (**FNR**) and True Negative Rate (**TNR**) are especially useful for imbalanced datasets
- In some problems it will be important to minimize the FNR, in other problems it will not be as important

$$\mathbf{TPR} = \frac{TP}{P} = \frac{TP}{TP + FN}$$

$$\mathbf{FPR} = \frac{FP}{N} = \frac{FP}{FP + TN}$$

$$\mathbf{FNR} = \frac{FN}{P} = \frac{FN}{FN + TP}$$

$$\mathbf{TNR} = \frac{TN}{N} = \frac{TN}{TN + FP}$$

Different evaluation metrics

- **Precision** (PRE), **Recall** (REC) and **F1**-score are central metrics in this course
- Recall (also known as *sensitivity* in medicine) is equivalent to TPR
- Precision seeks to measure the amount of TPs in relation to FPs

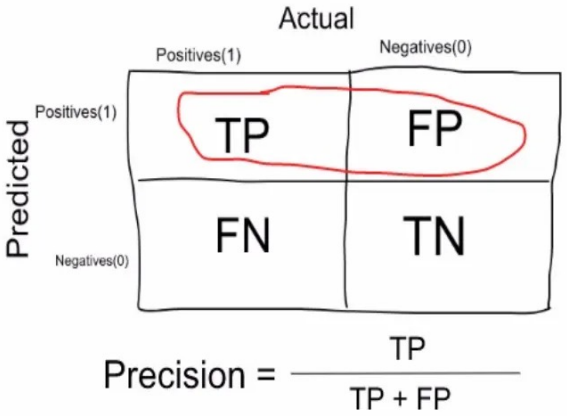
$$\mathbf{Recall} = \frac{TP}{TP + \mathbf{FN}} \quad \mathbf{Precision} = \frac{TP}{TP + \mathbf{FP}}$$

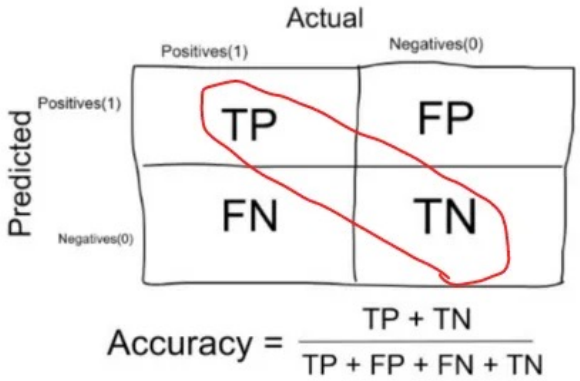
- Often, optimizing for recall might come at the cost of lowering the precision
- The F1-score is a metric that seeks to combine precision and recall

$$\mathbf{F1} = 2 \times \frac{\text{Precision} \times \text{Recall}}{\text{Precision} + \text{Recall}}$$

Performance metrics

- **Precision :** Measures the proportion of correctly predicted positive instances.

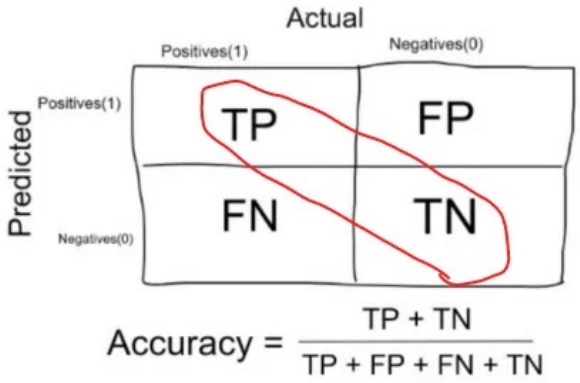


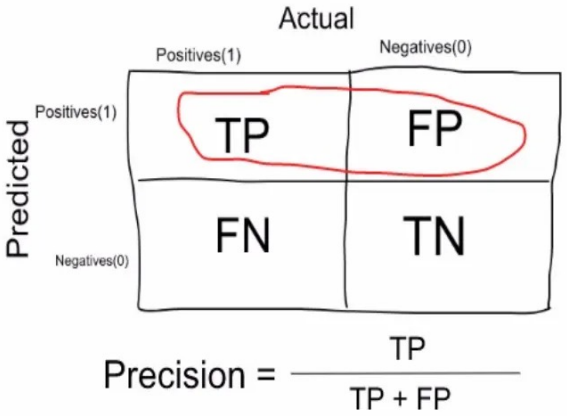


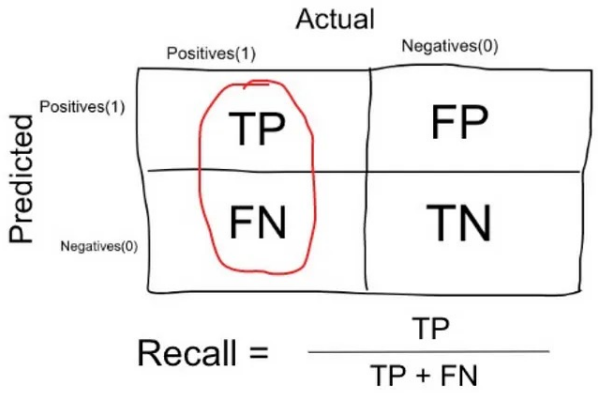
Source: https://medium.com/@MohammedS/performance-metrics-for-classification-problems-in-machine-learning-part-i-b085d432082b

Performance metrics

- **Recall:** Measures the proportion of actual positives correctly identified.



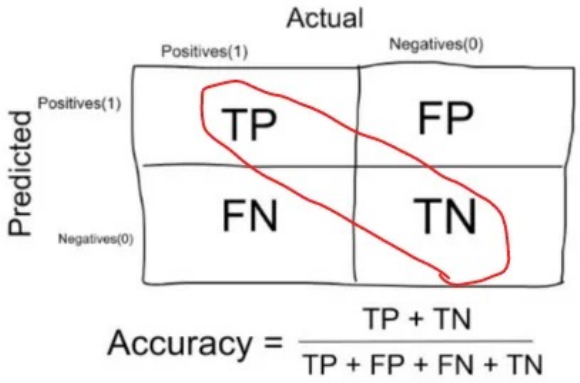


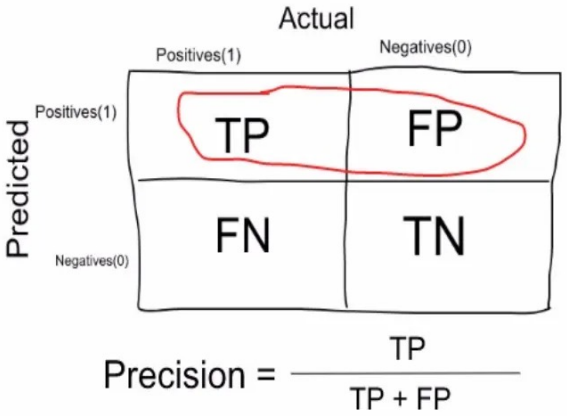


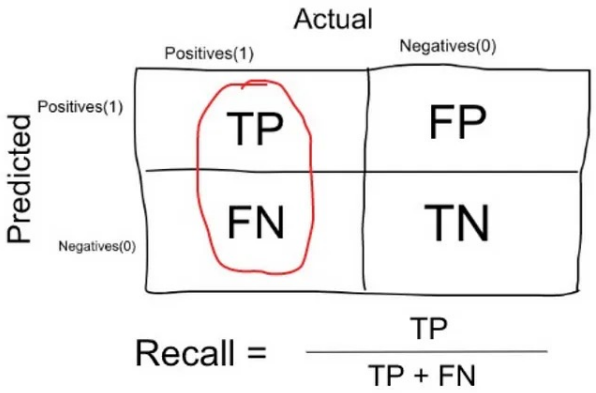
Source: https://medium.com/@MohammedS/performance-metrics-for-classification-problems-in-machine-learning-part-i-b085d432082b

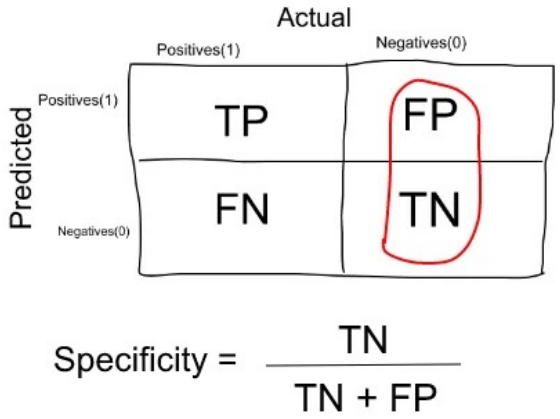
Performance metrics

- **Specificity:** Measures the proportion of actual negatives correctly identified.









Source: https://medium.com/@MohammedS/performance-metrics-for-classification-problems-in-machine-learning-part-i-b085d432082b

Performance metrics

- **F1 Score**: is a measure which represents both Precision (PRE) and Recall (REC).

$$F1 = 2\frac{PRE x REC}{PRE + REC}$$

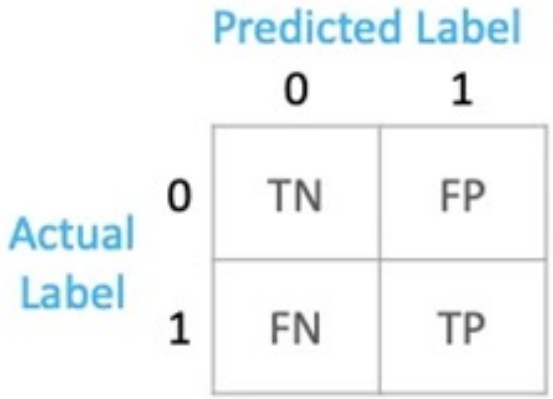
$$\text{F1 Score} = \frac{TP}{TP + \frac{1}{2}(FP + FN)}$$

Source: https://medium.com/@MohammedS/performance-metrics-for-classification-problems-in-machine-learning-part-i-b085d432082b

Scikit-learn confusion matrix

1. Default output

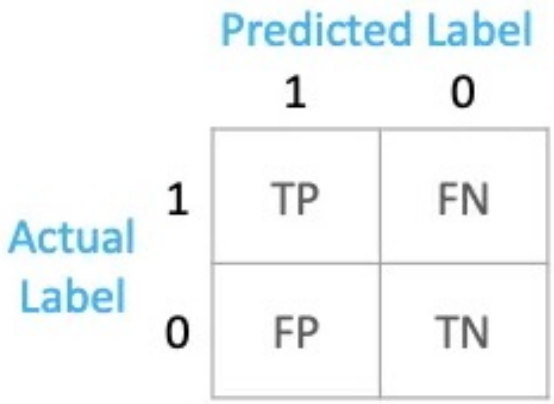
```
#1. Default output
confusion_matrix(y_true, y_pred)
```



Lecture's confusion matrix

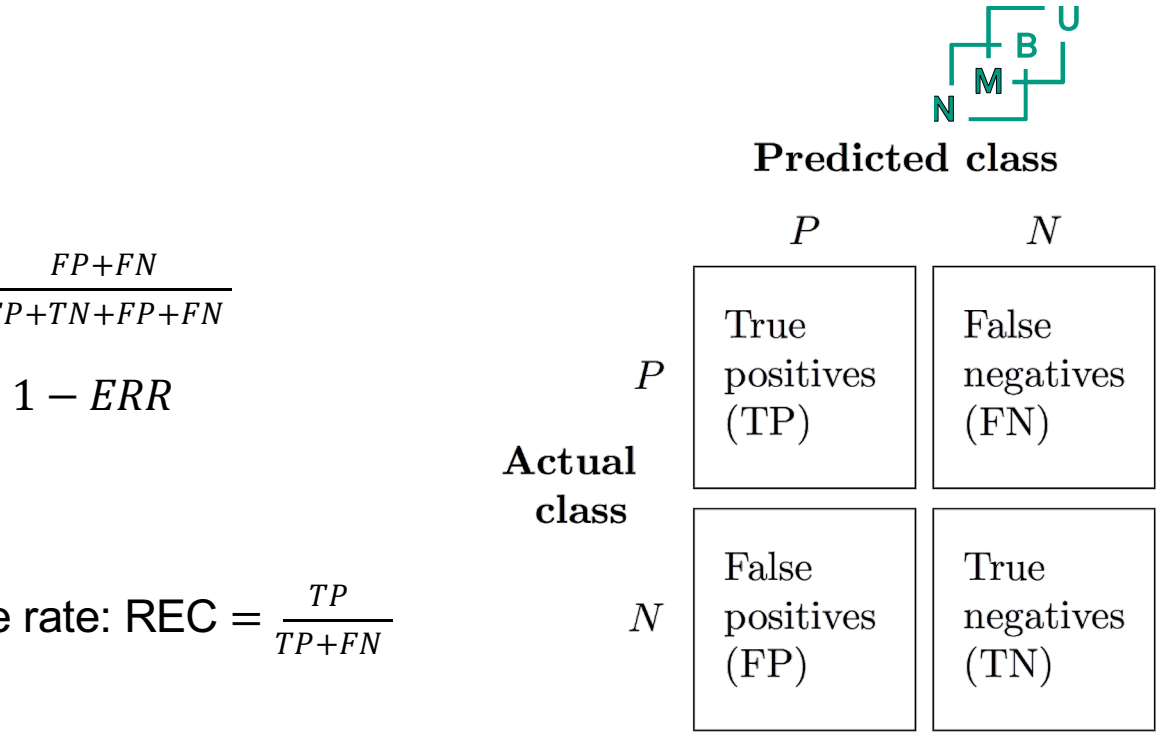
2. By adding the labels parameter, you can get the following output

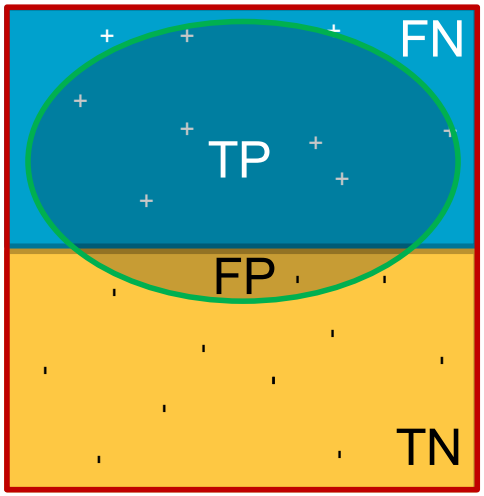
```
#2. Using labels parameter
confusion_matrix(y_true, y_pred, labels=[1,0])
```



Performance metrics

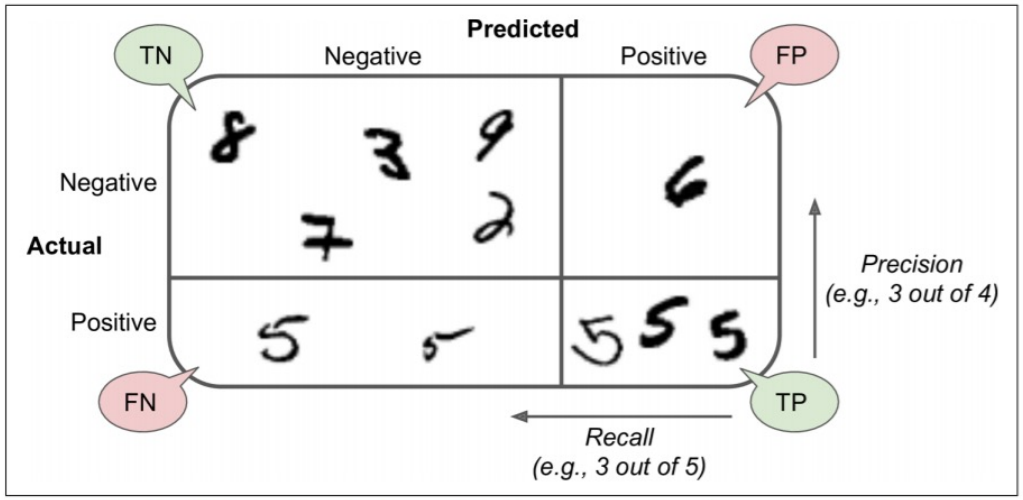
- Prediction error: ERR $= \frac{\#false}{\#object} = \frac{FP+FN}{TP+TN+FP+FN}$
- Accuracy: $\frac{\#true}{\#object} = \frac{TP+TN}{TP+TN+FP+FN} = 1 - ERR$
- Precision: PRE $= \frac{TP}{TP+FP}$
- Recall = Sensitivity = True positive rate: REC $= \frac{TP}{TP+FN}$
- F1 = 2 $\frac{PRExREC}{PRE+REC} = \frac{2TP}{2TP+FP+FN}$
- Specificity = True negative rate: $= \frac{TN}{TN+FP}$
- False discovery rate (low is good): FDR $= \frac{FP}{TP+FP}$
- False positive rate (1-Specificity): $= \frac{FP}{FP+TN}$
- Matthews correlation coefficient : $\mathrm{MCC} = \frac{TP \times TN - FP \times FN}{\sqrt{(TP+FP)(TP+FN)(TN+FP)(TN+FN)}}$





Performance evaluation for multiclass problems

- When evaluating multiclass models: Compute multiple “one-vs-all” assessments
- This is what one one-vs-all assessment looks like for handwritten digits: 5 vs. “not 5”



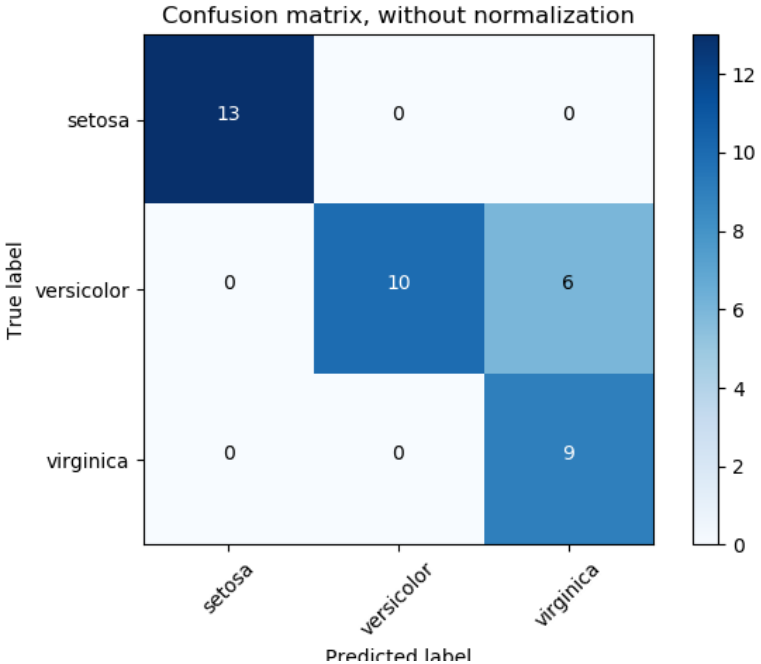
Performance evaluation for multiclass problems

- Can choose between **micro** and **macro** averaging:
  - Micro gives **equal weight** to every **sample**
  - Macro gives **equal weight** to every **class**

- Micro/macro averaging can be done for any performance metric

Performance evaluation for multiclass problems

- Confusion matrices for *n*-class multiclass problems have *n* x *n* dimensions



		Predicted class: P	Predicted class: N
Actual class	P	True positives (TP)	False negatives (FN)
Actual class	N	False positives (FP)	True negatives (TN)

Multiclass classification

No	Actual	Predicted	Match
1	Airplane	Airplane	✓
2	Car	Boat	X
3	Car	Car	✓
4	Car	Car	✓
5	Car	Boat	X
6	Airplane	Boat	X
7	Boat	Boat	✓
8	Car	Airplane	X
9	Airplane	Airplane	✓
10	Car	Car	✓

	precision	recall	f1-score	support
Aeroplane	0.67	0.67	0.67	3
Boat	0.25	1.00	0.40	1
Car	1.00	0.50	0.67	6
accuracy			0.60	10
macro avg	0.64	0.72	0.58	10
weighted avg	0.82	0.60	0.64	10

Per-Class F1 scores

Average F1 scores

Source: https://towardsdatascience.com/micro-macro-weighted-averages-of-f1-score-clearly-explained-b603420b292f

Multiclass classification – macro average

No	Actual	Predicted	Match
1	Airplane	Airplane	✓
2	Car	Boat	X
3	Car	Car	✓
4	Car	Car	✓
5	Car	Boat	X
6	Airplane	Boat	X
7	Boat	Boat	✓
8	Car	Airplane	X
9	Airplane	Airplane	✓
10	Car	Car	✓

	precision	recall	f1-score	support
Aeroplane	0.67	0.67	0.67	3
Boat	0.25	1.00	0.40	1
Car	1.00	0.50	0.67	6
accuracy			0.60	10
macro avg	0.64	0.72	0.58	10
weighted avg	0.82	0.60	0.64	10

Per-Class F1 scores

Average F1 scores

Label	Per-Class F1 Score	Macro-Averaged F1 Score
Airplane	0.67	$\frac{0.67 + 0.40 + 0.67}{3}$ = **0.58**
Boat	0.40	
Car	0.67	

Source: https://towardsdatascience.com/micro-macro-weighted-averages-of-f1-score-clearly-explained-b603420b292f

Multiclass classification – weighted average

No	Actual	Predicted	Match
1	Airplane	Airplane	✓
2	Car	Boat	X
3	Car	Car	✓
4	Car	Car	✓
5	Car	Boat	X
6	Airplane	Boat	X
7	Boat	Boat	✓
8	Car	Airplane	X
9	Airplane	Airplane	✓
10	Car	Car	✓

	precision	recall	f1-score	support
Aeroplane	0.67	0.67	0.67	3
Boat	0.25	1.00	0.40	1
Car	1.00	0.50	0.67	6
accuracy			0.60	10
macro avg	0.64	0.72	0.58	10
weighted avg	0.82	0.60	0.64	10

Per-Class F1 scores

Average F1 scores

Label	Per-Class F1 Score	Support	Support Proportion	Weighted Average F1 Score
Airplane	0.67	3	0.3	$(0.67 * 0.3) + (0.40 * 0.1) + (0.67 * 0.6)$ = **0.64**
Boat	0.40	1	0.1	
Car	0.67	6	0.6	
Total	-	10	1.0	

Source: https://towardsdatascience.com/micro-macro-weighted-averages-of-f1-score-clearly-explained-b603420b292f

Performance evaluation for multiclass problems

- There are almost as many evaluation metrics as there are problems
- Different people also use different names for the same metric
  - TPR/sensitivity/recall
  - Precision/positive predictive value
- Some are application specific
  - Computer vision: Mean Average Precision, Intersection over Union, etc.
  - Medicine: Diagnostic Odds Ratio
  - Unsupervised learning: Adjusted Rand Index

Performance evaluation for multiclass problems

- Some of my favourites include:
  - F1-score
  - Matthews Correlation Coefficient (MCC)
  - Balanced Accuracy

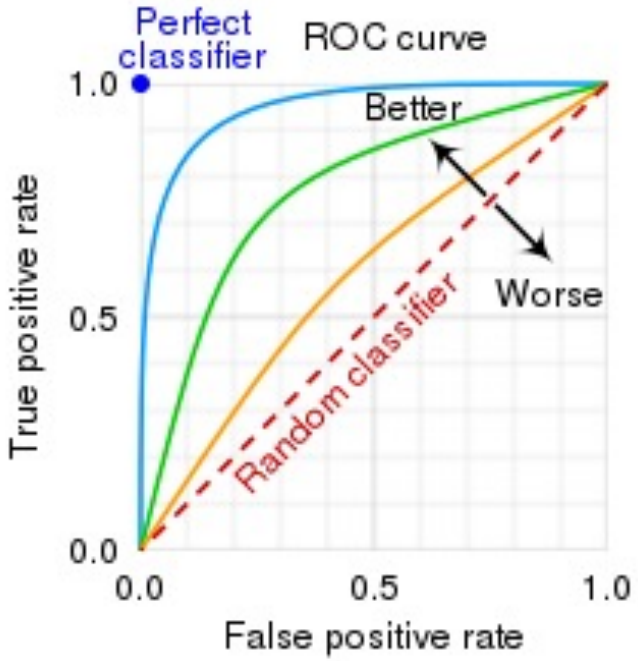
Performance evaluation for multiclass problems

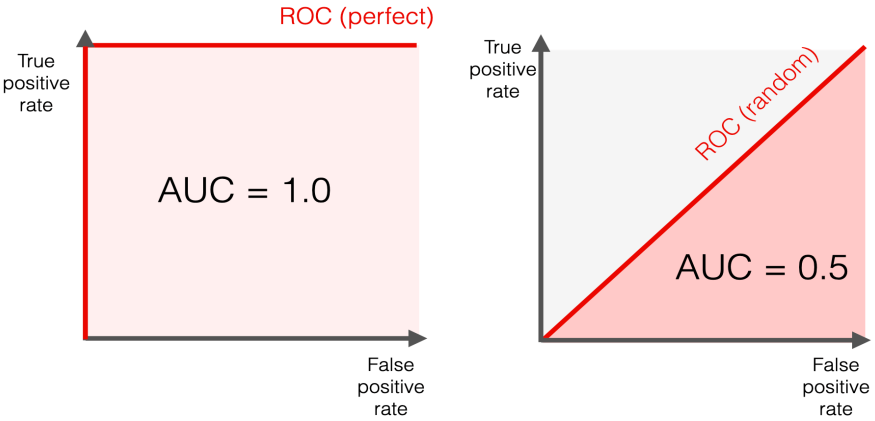
- Check out the Wikipedia page, or scikit-learn documentation for a good overview

		Predicted condition		Sources: [4][5][6][7][8][9][10][11][12] view • talk • edit	
	Total population $= P + N$	**Predicted Positive (PP)**	**Predicted Negative (PN)**	Informedness, bookmaker informedness (BM) $= TPR + TNR - 1$	Prevalence threshold (PT) $= \frac{\sqrt{TPR \times FPR} - FPR}{TPR - FPR}$
Actual condition	**Positive (P)** [a]	**True positive** (TP), hit[b]	**False negative** (FN), miss, underestimation	True positive rate (TPR), recall, sensitivity (SEN), probability of detection, hit rate, power $= \frac{TP}{P} = 1 - FNR$	False negative rate (FNR), miss rate type II error [c] $= \frac{FN}{P} = 1 - TPR$
	Negative (N)[d]	**False positive** (FP), false alarm, overestimation	**True negative** (TN), correct rejection[e]	False positive rate (FPR), probability of false alarm, fall-out type I error [f] $= \frac{FP}{N} = 1 - TNR$	True negative rate (TNR), specificity (SPC), selectivity $= \frac{TN}{N} = 1 - FPR$
	Prevalence $= \frac{P}{P + N}$	Positive predictive value (PPV), precision $= \frac{TP}{PP} = 1 - FDR$	False omission rate (FOR) $= \frac{FN}{PN} = 1 - NPV$	Positive likelihood ratio (LR+) $= \frac{TPR}{FPR}$	Negative likelihood ratio (LR−) $= \frac{FNR}{TNR}$
	Accuracy (ACC) $= \frac{TP + TN}{P + N}$	False discovery rate (FDR) $= \frac{FP}{PP} = 1 - PPV$	Negative predictive value (NPV) $= \frac{TN}{PN} = 1 - FOR$	Markedness (MK), deltaP (Δp) $= PPV + NPV - 1$	Diagnostic odds ratio (DOR) $= \frac{LR+}{LR-}$
	Balanced accuracy (BA) $= \frac{TPR + TNR}{2}$	F_1 score $= \frac{2\,PPV \times TPR}{PPV + TPR} = \frac{2\,TP}{2\,TP + FP + FN}$	Fowlkes–Mallows index (FM) $= \sqrt{PPV \times TPR}$	Matthews correlation coefficient (MCC) $= \sqrt{TPR \times TNR \times PPV \times NPV} - \sqrt{FNR \times FPR \times FOR \times FDR}$	Threat score (TS), critical success index (CSI), Jaccard index $= \frac{TP}{TP + FN + FP}$

Receiver Operator Curve and Area Under Curve

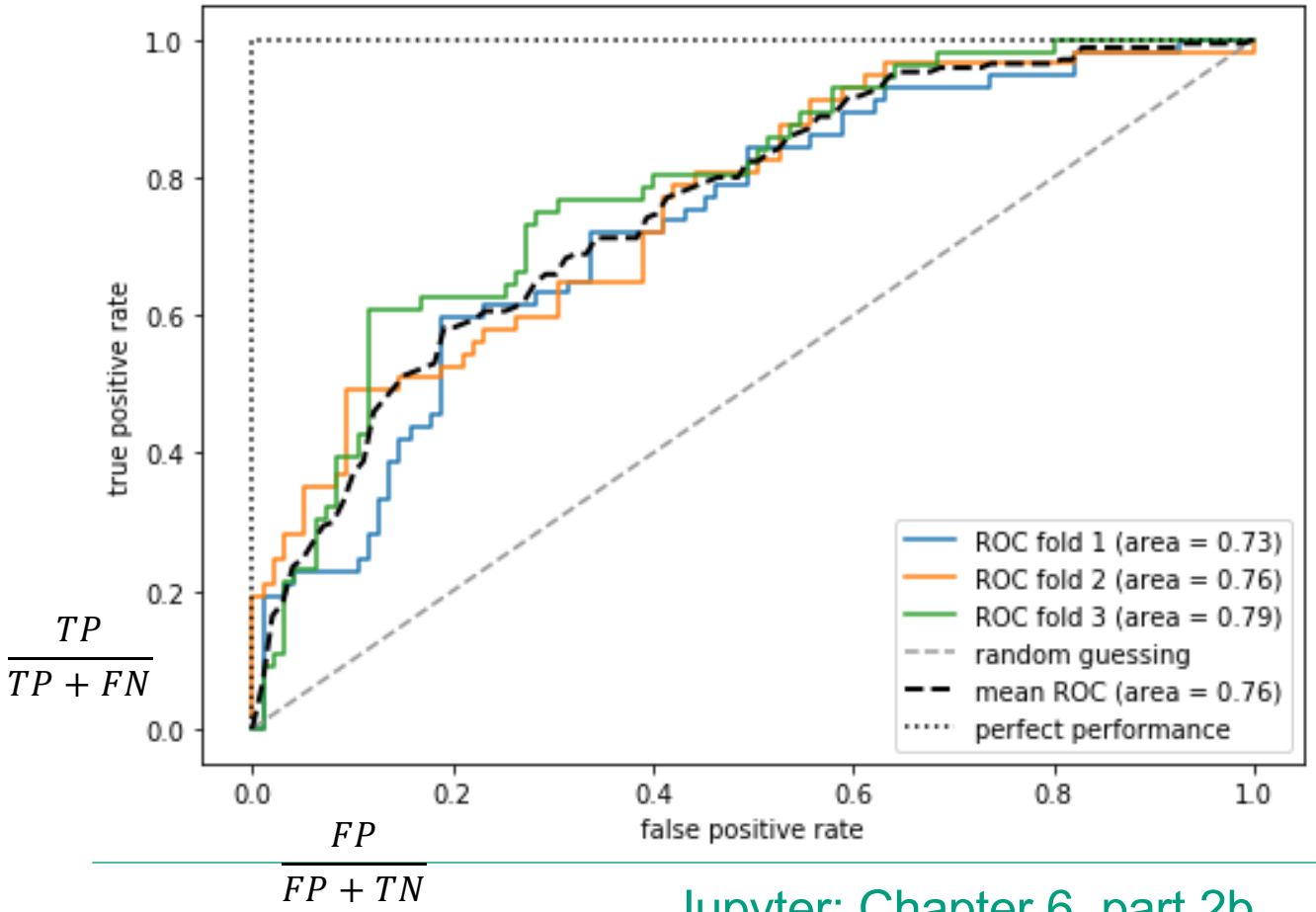
- ROC-AUC: Area Under the Receiver Operating Characteristic Curve
- Quantitative measure of the overall classifier performance at all possible thresholds
- Illustrates the trade-off between FPR and TPR
- To compute the ROC classifiers must output a probabilistic output which can be thresholded
- The diagonal curve is the performance of random guessing (worst possible score)
- The blue curve is better





Receiver Operating Curves (ROC)

ROC shows a model's ability to **distinguish between classes** at all thresholds.



$$\frac{TP}{TP + FN}$$

$$\frac{FP}{FP + TN}$$

AUC (Area Under the Curve):
A single number that summarizes the **overall performance** of a model on the ROC curve.

- **AUC = 1.0** → perfect separation
- **AUC = 0.5** → no better than random guessing

Online illustration:
http://www.navan.name/roc/

- Inspect at top of peaks
- Inspect at middle point
- Move around

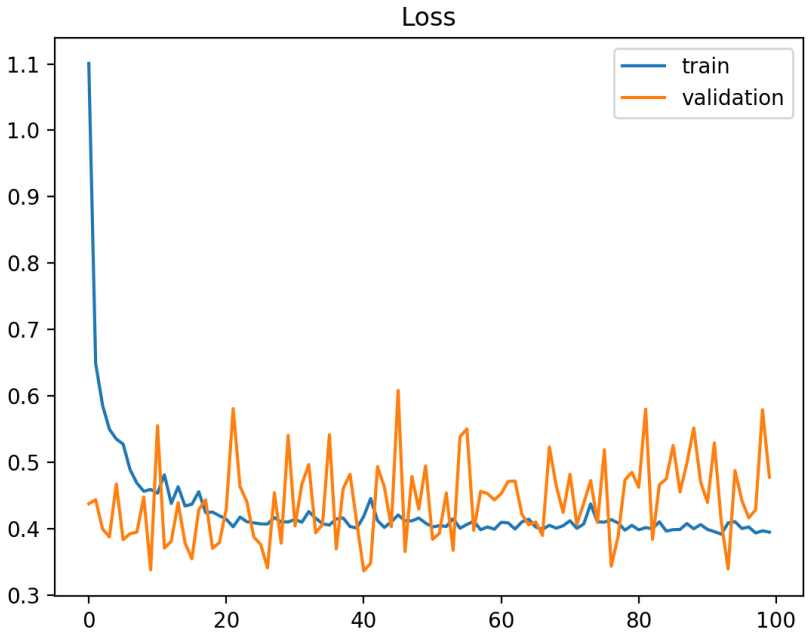
Jupyter: Chapter 6, part 2b

Metrics worth knowing about

- You don’t need to memorize the formulas for all the evaluation metrics that have been presented in this lecture for the exam
- BUT: You **should memorize** the following:
  - Confusion matrix
  - Precision, Recall and F1-score
  - Concept of an ROC curve, and the ROC-AUC metric
  - Difference between micro and macro averaging when computing multiclass metrics

Class activity

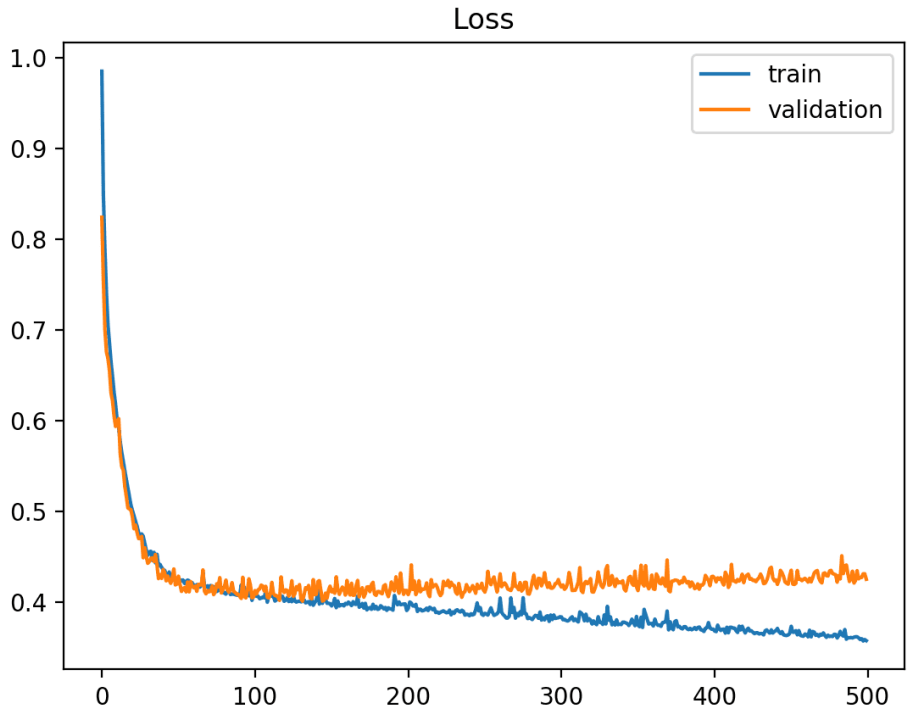
- What is going on?



Source: https://machinelearningmastery.com/learning-curves-for-diagnosing-machine-learning-model-performance/

Class activity

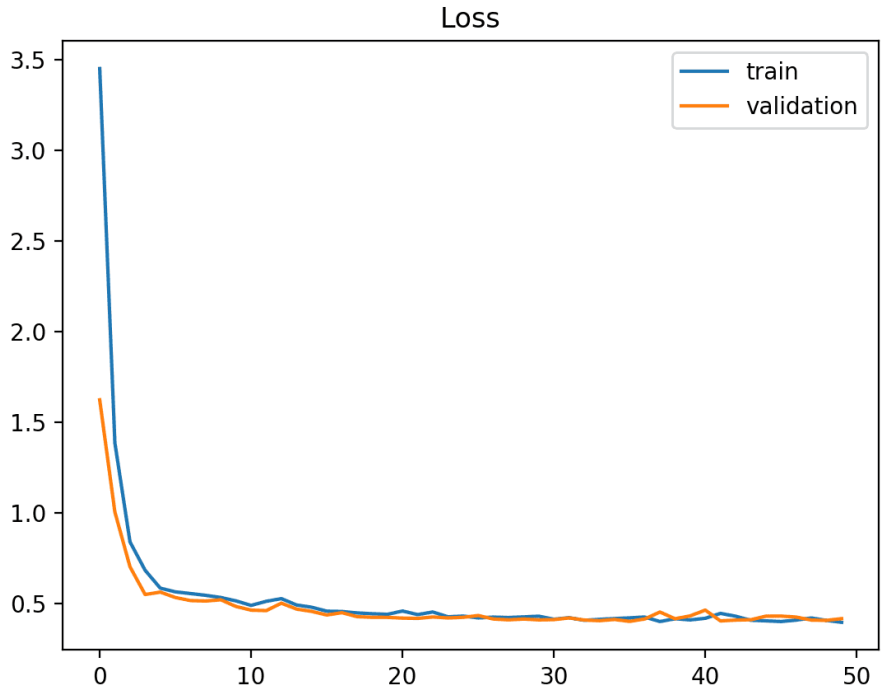
- What is going on?



Source: https://machinelearningmastery.com/learning-curves-for-diagnosing-machine-learning-model-performance/

Class activity

- What is going on?



Source: https://machinelearningmastery.com/learning-curves-for-diagnosing-machine-learning-model-performance/

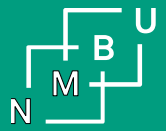
Summary

- Pipelines
- Validation / cross-validation
- Learning and validation curves
- Grid search (and friends)
- Nested cross-validation
- Performance metrics

- Series of modules for data transformation and modelling. Acts as a single model in scikit-learn.
- Splitting of the data to reduce overfitting and search for parameter values. Many sets of “new” data”.
- Assessment of model bias and variance, performance along the number of training samples/different parameter values.
- Well-organised search for the optimal (combination of) parameter values.
- Estimation of future performance when tuning parameters on the current data.
- Various measures of success, dealing with multiple classes and class imbalance.

Examples on metrics with scikit-learn

- metrics_example.ipynb
  - Example of how to compute evaluation metrics with scikit-learn


U
B
M
N